공부 내용 정리 (3장)

3.1 트랜스포머 아키텍처

트랜스포머: 자연어를 처리하기 위해 만들어진 딥러닝 모델 구조

트랜스포머 특징

-self-attention을 사용해 문맥을 반영함

-RNN처럼 순차적인 상태 전달에만 의존하지X

-여러 토큰의 관계를 병렬 계산할 수 있음

-멀리 떨어진 단어 사이의 관계도 직접 계산할 수 있음

트랜스포머의 전체 구조

Encoder: 입력 문장에 있는 각 토큰이 다른 토큰과 어떤 관계를 가지는지 파악하고, 문맥이 반영된 벡터 표현 생성

Decoder: 이전에 생성한 토큰과 Encoder의 정보를 참고해 다음 토큰을 예측

세 가지 트랜스포머 구조

1. Encoder-only: 입력 문장의 의미 이해 -Bert
2. Decoder-only: 다음 토큰을 예측하며 생성 - GPT
3. Encoder-Decoder: 입력을 처리해 새로운 출력 생성 -T5,BART

3.2 인코더

인코더는 입력 토큰을 받아 문맥 정보가 포함된 벡터로 변환하는 부분

3.2.1 셀프 어텐션

Attention: 어떤 토큰을 처리할 때 다른 토큰의 정보를 얼마나 참고할지 계산하는 방법

Self-Attention: 문장 내 토큰끼리 관계를 계산

Query (Q) - 어떤 정보를 찾는지 표현
Key (K) - 어떤 정보를 가지고 있는지 비교
Value (V) - 실제 전달할 정보

Multi-Head Attention: 하나의 Attention만 사용하는 대신 여러 개의 Attention Head를 사용하면 서로 다른 표현 공간에서 관계를 학습 가능

3.2.2 피드 포워드 층

Attention은 토큰 간의 정보를 결합하는 역할을 함. 그런데 정보를 결합하기만 해서는 표현력이 충분하지 않기 때문에 Feed Forward 층을 통해 각 토큰의 정보를 추가로 변환함

3.2.3 층 정규화 추가하기

Residual Connection: 블록의 입력을 블록의 출력에 더하는 방식

Layer Normalization: 각 토큰의 특징값을 정규화하는 방법

Post-LN과 Pre-LN

Post-LN: 먼저 변환과 잔차 연결을 적용하고, 그 다음 정규화하는 방식
Pre-LN: 먼저 정규화한 다음 변환을 수행하고 입력을 더함

3.2.4 위치 임베딩

필요한 이유: Attention은 각 토큰 사이의 관련성을 계싼하지만, 위치 정보가 따로 제공되지 않으면 토큰의 순서를 충분히 표현할 수 없음

Token Embedding과 Position Embedding

Token Embedding: 토큰 자체의 정보를 벡터로 변환
Position Embedding: 토큰의 위치 정보를 벡터로 변환

3.2.5 분류 헤드 추가하기

Encoder는 입력 토큰의 문맥 표현을 만든다. 그런데 이 표현만으로는 바로 감정 분석처럼 특정 클래스의 정답을 출력하지X 그래서 Encoder위에 별도의 분류 레이어를 추가함. 이것이 Classification Head

3.3 디코더

텍스트를 생성하는 역할
미래 토큰을 보지 못하게 Mask를 사용함

Encoder와 Decoder 비교

구분	Encoder	Decoder
주요 목적	입력 표현 생성	텍스트 생성

구분	Encoder	Decoder
Self-Attention	일반적으로 양방향	Causal Mask 사용
미래 토큰	양방향 문맥 활용 가능	참고 불가
대표 모델	BERT	GPT

3.2 트랜스포머 유니버스

3.4.1 트랜스포머 가계도
Encoder/Decoder/Encoder-Decoder

3.4.2 인코더 유형

인코더 계열 모델은 자연어 이해 작업에 자주 사용됨 -BERT

두 가지 사전 학습 목적

MLM: 문장의 일부 토큰을 가리고 원래 토큰을 예측하게 하는 방식
NSP: 두 문장이 원래 이어지는 문장인지 예측하는 방식

RoBERTa: BERT의 사전 학습 방법을 개선한 모델

XLM: 여러 연어에 걸친 언어 표현을 학습하는 모델

ALBERT: BERT의 매개변수 수를 효율적으로 줄이도록 설계한 모델

ELECTRA: BERT처럼 가려진 토큰을 복원하는 방식 대신에 입력의 토큰이 원래 토큰인지 다른 토큰으로 대체된 것인지 판별하는 사전 학습 방식 사용

3.2.3 디코더 유형

GPT: 앞에서 나온 토큰을 보고 다음 토큰을 예측하는 방식으로 사전 학습함

GPT-2: GPT 계열의 확장 모델, 더 큰 규모의 모델과 데이터로 언어 생성 능력을 발전시킴

CTRL: 제어 코드를 사용해 생성할 텍스트의 유형이나 스타일을 조절하도록 설계된 언어 모델

GPT-3: 더 확장

3.4.4 인코더 - 디코더 유형

입력을 처리한 결과를 바탕으로 새로운 텍스트를 생성함
활용 분야: 번역, 요약, 질문 답변 등

T5: 다양한 NLP 작업을 Text-to-Text 형태로 통일해서 다루는 모델

BART: 입력 텍스트를 손상시킨 후 원래 텍스트를 복원하도록 학습하는 모델

M2M -100: 다국어 변역을 위해 개발된 모델

BigBird: 긴 시퀀스를 처리하기 위한 모